Sorting Algorithms

Counting Sort

Core Idea

Instead of comparing elements like all previous sorts, **count how many times each value appears** in the array. Then use those counts to place each element directly in its correct position. No comparisons at all — just counting and placing.

The Three Phases

- **Count** — count the frequency of each element
- **Accumulate** — convert counts into positions (prefix sum)
- **Place** — use the positions to build the sorted output array

---

Steps

1. Find the **max value** in the array to know the size of the count array
2. Create a **count array** of size `max + 1`, initialized to all zeros
3. Walk through the input and **increment count** at the index matching each value
4. Convert count array into a **prefix sum** — each index now holds the position where that value should end
5. Walk the input array **backwards**, place each element at its correct position in the output
6. Copy output back to original array

Visualizing It

`[4, 2, 2, 8, 3, 3, 1]`

Phase 1 — Count:

```
index:  0  1  2  3  4  5  6  7  8
count: [0, 1, 2, 2, 1, 0, 0, 0, 1]
```

- 1 appears once, 2 appears twice, 3 appears twice, 4 once, 8 once

Phase 2 — Prefix Sum (accumulate):

```
index:  0  1  2  3  4  5  6  7  8
count: [0, 1, 3, 5, 6, 6, 6, 6, 7]
```

- Each index now tells you the **ending position** of that value in the output

Phase 3 — Place (walk backwards):

```
Input:  [4, 2, 2, 8, 3, 3, 1]
Output: [_, _, _, _, _, _, _]
```

- 1 → goes to position `count[1]-1 = 0` → output[0] = 1, decrement count[1]
- 3 → goes to position `count[3]-1 = 4` → output[4] = 3, decrement count[3]
- 3 → goes to position `count[3]-1 = 3` → output[3] = 3, decrement count[3]
- 8 → goes to position `count[8]-1 = 6` → output[6] = 8, decrement count[8]
- 2 → goes to position `count[2]-1 = 2` → output[2] = 2, decrement count[2]
- 2 → goes to position `count[2]-1 = 1` → output[1] = 2, decrement count[2]
- 4 → goes to position `count[4]-1 = 5` → output[5] = 4, decrement count[4]

```
Output: [1, 2, 2, 3, 3, 4, 8] ✅
```

Key Characteristics

- **Stable** — walking backwards and decrementing preserves original order of equal elements
- **Not in-place** — needs a separate count array and output array
- **Non comparison based** — never compares two elements directly

---

Time & Space Complexity

Case	Time	Why
Best	O(n + k)	One pass to count, one pass to place
Average	O(n + k)	Always the same two passes
Worst	O(n + k)	Always the same two passes

Space: O(n + k) — count array of size k, output array of size n

> `k` = range of input values (max - min). `n` = number of elements.

The Optimization

If the range `k` is much larger than `n`, Counting Sort becomes inefficient.

For example sorting `[1, 50000]` — only 2 elements but needs a count array of 50000. In such cases **offset the minimum** — subtract the min value from all elements so the count array only covers the actual range, not from zero.

When to Use vs When to Avoid

Use When	Avoid When
Range `k` is small and known	Range is huge (floating points, large integers)
Integers or discrete values only	Data has negative numbers (needs extra handling)
Stability matters	Memory is constrained

Why It's Asked in Interviews

It breaks the **O(n log n) comparison sort barrier** and achieves O(n) — interviewers use it to test if you know the theoretical limits of sorting. It also directly connects to **Radix Sort** which builds on top of Counting Sort as its core subroutine.

Implementation

```
idx = prefix_sum[ele] - 1
Why are we doing -1?
```

Because `prefix_sum[ele]` stores a **count** (how many elements ≤ ele), but you need an **index** (0-based).

Simple example

```
nums = [1, 2, 3]
prefix_sum[3] = 3   # there are 3 elements ≤ 3
```

But valid indices are `0, 1, 2` — so index of `3` should be `2`, not `3`.

```
prefix_sum[ele] - 1 = 3 - 1 = 2  ✅
```

One more with duplicates

```
nums = [1, 2, 2, 3]
prefix_sum[2] = 3   # there are 3 elements ≤ 2  (one 1, two 2
s)
```

The two `2s` should land at indices `2` and `1`:

```
First  2: idx = 3 - 1 = 2, then decrement → prefix_sum[2] = 2
Second 2: idx = 2 - 1 = 1, then decrement → prefix_sum[2] = 1
```

In short: `-1` is just converting from 1-based count → 0-based index.

Why Reverse?

```
for i in reversed(range(n)):
Why are we going in reverse?
```

To maintain **stability** — equal elements should keep their original relative order from `nums`.

What stability means

```
nums = [4a, 2, 4b]   # 4a and 4b are both 4, subscript shows
original order
```

A stable sort should produce `[2, 4a, 4b]` — `4a` before `4b`, same as original.

What happens going forward vs reverse

```
prefix_sum[4] = 3  # both 4s will go to indices 2 and 1
```

python

```
# ❌ Forward — processes 4a first
i=0: ele=4a → output[2]=4a, prefix_sum[4]=2
i=2: ele=4b → output[1]=4b, prefix_sum[4]=1
result = [2, 4b, 4a]  # 4b before 4a — WRONG order!

# ✅ Reverse — processes 4b first
i=2: ele=4b → output[2]=4b, prefix_sum[4]=2
i=0: ele=4a → output[1]=4a, prefix_sum[4]=1
result = [2, 4a, 4b]  # 4a before 4b — correct order!
```

Why reverse fixes it

- Reverse processes the **last duplicate first**, placing it at the **highest available index**

- Then the **first duplicate** naturally lands at the **lower index**
- So left-to-right order in `nums` is preserved in `output`

In short: reverse iteration ensures the first occurrence of a duplicate in `nums` ends up to the left of later occurrences in `output`.

For plain numbers, stability doesn't matter — `4a` and `4b` are identical, so their order is irrelevant.

🌟 So why bother with stability at all?

Counting sort is rarely used alone in real problems. It's used as a **building block inside Radix Sort**, where stability is **critical**.

```
# Sort these numbers by individual digits using counting sort
at each step:
nums = [121, 123, 421, 423]

# Sort by 1s digit:
[121, 421, 123, 423]   # stable: 121 before 421, 123 before 4
23

# Sort by 10s digit (all same, stability preserves previous o
rder):
[121, 421, 123, 423]

# Sort by 100s digit:
[121, 123, 421, 423]  # ✅ correct! stability from step 1 car
ried through
```

If counting sort wasn't stable, **radix sort would break**.

Can counting sort be used for strings?

Not directly — counting sort requires:

1. Elements must be **integers** (or mappable to integers)
2. Range of values must be **small and known**

But strings can work if you sort **character by character**:

```
# Sort by each character position — this is exactly how radix
sort handles strings
["bad", "ace", "abc"]
→ sort by 3rd char → ["abc", "ace", "bad"]
→ sort by 2nd char → ["abc", "ace", "bad"]
→ sort by 1st char → ["abc", "ace", "bad"]  ✅
```

In short: stability is irrelevant for standalone number sorting, but essential when counting sort powers radix sort — which works on both numbers and strings.

🌟 Negative Numbers

Standard counting sort **breaks with negative numbers** because it uses values as array indices, and negative indices don't work.

python

```
nums = [-3, 1, 2]
count_freq = [0] * (max_val + 1)  # max_val = 2, so size 3
count_freq[-3] += 1  # ❌ -3 as index — in Python this wraps
around to end of array!
```

Fix — shift everything by `min_val`

Situation	Counting Sort?
Small integer range	✅ ideal — O(n+k)
Large range, few elements	❌ wastes memory
Floats or decimals	❌ can't use as index
Strings directly	❌ needs radix sort approach